

Fermion TeX Engine システム説明書

このプロジェクトは何を、どういう仕組みで「いじって」いるのか

TeX DOM Runtime プロジェクト

2026 年 7 月 3 日

概要

Fermion TeX Engine は、組版システム LaTeX を「編集のたびに全体を作り直すバッチ処理」から「文書を常駐させ、直した部分だけを数ミリ秒で組み直すランタイム」に作り替えたものです。本書はプロジェクト全体の設計を一冊で説明します。特に力を入れたのは「何をいじって、何をいじっていないのか」の完全な目録です。本エンジンは LaTeX 本体（プログラム・組版アルゴリズム・フォント処理）には 1 バイトも手を加えていません。外から注入しているのは、(1) OS の fork 機能を橋渡しする 71 行の C コード、(2) 組版結果を写し取る Lua プログラム、(3) 動作を変えずに記録だけを行う約 40 行の TeX マクロ、の 3 種類だけです。この「注入物の少なさ」こそが、プレビューが本物の PDF と一致することの根拠になっています。

目次

1	この説明書について	2
2	一言でいうと何をするソフトか	2
3	予備知識 — TeX はどうやって PDF を作るか	2
3.1	組版の流れ	2
3.2	LuaTeX の特別なところ	3
4	全体アーキテクチャ	3
4.1	登場するプロセス	3
4.2	責任分担の原則	4
5	何を「いじって」いるのか — 注入物の完全目録	4
5.1	いじって「いない」もの	4
5.2	注入物 1: C シム tdomfork.c (71 行)	5
5.3	注入物 2: Lua デーモン daemon.lua (約 900 行)	5
5.4	注入物 3: TeX マクロのシム (約 40 行)	5

5.5	注入物 4：ページビルダーの「休眠」設定	6
5.6	安全性について — <code>--shell-escape</code> の意味	6
6	心臓部 1 — fork チェックポイント	6
6.1	fork は「TeX の完全なセーブデータ」である	6
6.2	凍結のからくり	7
6.3	編集 1 回で起きること	7
6.4	プロセス数の管理	8
7	心臓部 2 — 組版結果の収穫	8
7.1	本流の上で組み、写し取る	8
7.2	run 分割の契約 — 座標を全部送らずに済む理由	8
7.3	収穫される JSON の中身	8
8	心臓部 3 — ページ割り付け（規則書の移植）	9
8.1	なぜ Node 側で改ページするのか	9
9	描画層 — ブラウザで「印刷と同じ絵」を出す	9
9.1	フォントは TeX が使った実ファイルを配る	9
9.2	ブラウザの整形機能をすべて切る	9
9.3	数式の Type1 問題と「双子」フォント	10
9.4	2 層戦略 — 即時グリフ+精密チャンク	10
10	打鍵から画面まで — 時系列	10
11	正しさの保証	11
12	コードマップと API	11
12.1	リポジトリの地図	11
12.2	3 世代のバックエンド	11
12.3	HTTP API	12
13	動かし方と動作条件	12
14	用語集	12

1 この説明書について

■**対象読者** プログラミングの一般的な知識（プロセス、サーバー、ブラウザ程度の語彙）はあるが、TeX の内部には詳しくない方を想定しています。TeX の予備知識は第 3 章で必要な分だけ説明します。

■関連文書

- docs/01~07 (Markdown) — 実装の完全詳解。ソースコードと突き合わせて読むための技術資料で、開発中に踏んだ罫の記録（第 6 章）を含みます。
- docs/implementation-report.pdf — v3.2 の改良（プレビューと PDF の完全一致化）に絞ったやさしい読み物。
- 本書 — プロジェクト全体の仕組みを一冊で説明する「説明書」。上の二つの中間の詳しさです。

2 一言でいうと何をするソフトか

LaTeX（ラテフ）は論文・技術書の定番組版システムです。仕上がりは印刷品質ですが、一文字直しただけでも文書全体を最初から処理し直すため、結果を見るまで数秒~数十秒かかります。

本エンジンはこれを次のように変えます。

	従来の LaTeX	Fermion TeX Engine
処理の形	使い捨てのバッチ処理	常駐プロセス(起動しっぱなし)
編集 1 回のコスト	文書全体の再処理	直した段落 1 つ分の組版 (4~14 ミリ秒)
画面反映まで	数秒~	約 0.03 秒 (打鍵ごと)
見た目の計算	LaTeX 本体	同じ LaTeX 本体 (無改変)

つまり「Word のような即時性」と「LaTeX の印刷品質」の両立です。脚注・図表の自動配置・目次・相互参照・日本語 (luatexja) まで、編集した瞬間にライブで更新されます。

3 予備知識 — TeX はどうやって PDF を作るか

本エンジンが「どこに手を入れたか」を理解するには、まず TeX が内部で何をしているかをざっくり知る必要があります。

3.1 組版の流れ

TeX が原稿（テキストファイル）から PDF を作るまでは、おおよそ次の段階を踏みます。

1. **読み取り**：原稿を 1 文字ずつ読み、マクロ (`\section` など) を展開する。
2. **段落の組版**：段落分の文字がたまると、有名な Knuth–Plass アルゴリズムが「どこで行を折り返すと最も美しいか」を段落全体で最適化する。ハイフネーション、カーニング（字間の微調整）、リガチャ（合字）もここで確定する。
3. **垂直リストへの追加**：出来上がった各行は「箱（ボックス）」として、**メイン垂直リスト**と呼ばれる縦一列の並びに追加されていく。箱と箱の間には「グルー（伸び縮みできる空白）」や「ペナルティ（ここで改ページすると減点いくつ、という印）」が挟まる。
4. **ページビルダー**：垂直リストがページの高さに達すると、ペナルティやグルーの情報から最良の改ページ位置を選び、1 ページ分を切り出す。
5. **出力ルーチン**：切り出された 1 ページ分に、脚注・図表（フロート）・ページ番号などを合成して、最終的なページの箱を作る（LaTeX ではここが `\output` ルーチンと呼ばれる仕組みで実装されている）。
6. **PDF 書き出し**：ページの箱の中の文字や線を PDF 命令に変換して書き出す。

覚えておいてほしいキーワードは「**ノード**」です。TeX の内部では、文字 1 つ、空白 1 つ、改ページ禁止の印 1 つが、それぞれ「ノード」というデータとしてリスト状につながっています。組版とは「ノードの列を作って寸法を確定させる」作業に他なりません。

3.2 LuaTeX の特別なところ

本プロジェクトが土台に選んだ **lualatex (LuaHBTeX)** は、TeX の現行エンジンのひとつで、内部にスクリプト言語 **Lua** を積んでいます。重要なのは次の 2 点です。

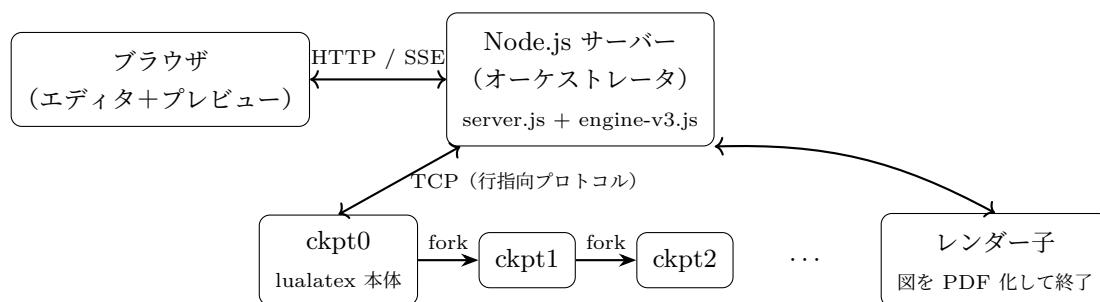
- Lua のプログラムから、**組版済みのノードの列を直接読める**（どの文字がどの座標に置かれたか、グルーが実際に何 mm に伸びたか、まで見える）。
- Lua から OS の共有ライブラリを読み込める（後述の C シムはこれで入れる）。

つまり LuaTeX は最初から「外から中身をのぞく公式の窓」を持っています。本エンジンはこの窓だけを使い、窓の内側（組版計算そのもの）には触れません。

4 全体アーキテクチャ

4.1 登場するプロセス

システムは 3 つの世界からできています：ブラウザ（画面）、Node.js のサーバー（司令塔）、そして lualatex のプロセスの群れ（組版の実体）です。



ブラウザ 左にテキストエディタ、右にプレビュー。編集すると差分だけをサーバーに送り、返ってきた「描画命令の差分」で該当ページだけを描き替えます。中身は徹底して薄く、組版の知識は持ちません。

Node.js サーバー（オーケストレータ） 文書の「真実」を握る側。ソース全文、ブロック分割、ラベル表、ページ、フォント台帳を保持し、lualatex プロセス群に仕事を割り振ります。ページへの割り付け（第 8 章）もここで行います。

lualatex プロセスの群れ 起動時に 1 個だけ本物の lualatex を立ち上げ、あとは fork（自己複製）で増えます。各プロセスが「文書を途中まで処理した状態」の生きたセーブポイントです（第 6 章）。

4.2 責任分担の原則

- 文字の並び・行の折り返し・あらゆる寸法の決定 — lualatex（無変更の本物）。
- どのブロックを組み直すかの判断、ページへの割り付け、差分配信 — Node.js。ただし割り付けは TeX の規則書をそのまま移植したもので、独自の判断はしません。
- 描画 — ブラウザ。ただし TeX が使ったフォントファイルそのものを使い、ブラウザ独自の文字詰め機能はすべて無効化します（第 9 章）。

5 何を「いじって」いるのか — 注入物の完全目録

本プロジェクトの正確な要約は次の一文です。

「lualatex が計算する内容」は一切変えず、「いつ・どの単位で計算するか」と「結果をどこへ出すか」だけを差し替えた。

5.1 いじって「いない」もの

先に、無変更のものを列挙します。ここが本エンジンの信頼性の土台です。

無改変のもの	意味
lualatex (LuaHBTeX) のバイナリと C ソース	TeX Live のものをそのまま起動するだけ
Knuth–Plass 行分割・数式組版・ハイフネーション	段落や数式の見た目を決める計算はすべて原作者のコードが実行
フォント処理 (luaotfload / HarfBuzz)	リガチャ・カーニング・字形選択は従来どおりエンジン内で確定
LaTeX カーネルとフォーマット パッケージ群 (amsmath, tikz, booktabs, luatexja, ...)	latex.ltx 無改変、lualatex.fmt をそのままロード 無改変
PDF エクスポート	「PDF を保存」は素の lualatex の 2 回実行。ライブ表示との一致検算にもなる

5.2 注入物 1：C シム tdomfork.c (71 行)

LuaTeX の Lua には、プロセスを複製する `fork` を呼ぶ標準手段がありません（同梱の FFI ライブラリは Apple Silicon 非対応）。そこで `fork` / `waitpid` / `_exit` / `getpid` / 「子プロセスの後始末を OS に任せる設定」の 5 つの OS 機能を Lua に橋渡しするだけの共有ライブラリを C で書きました。全 71 行で、組版には一切関与しません。初回起動時にオーケストレータが `cc` で自動コンパイルします。

5.3 注入物 2：Lua デーモン daemon.lua (約 900 行)

lualatex の中に読み込まれる Lua プログラムで、仕事は 4 つです。

1. **通信**：オーケストレータと TCP ソケットでつながり、命令（「このブロックを組め」など）を待ち受ける。
2. **凍結**：命令待ちの間、プロセスを眠らせる（＝セーブポイントとして保存）。
3. **収穫**：組版済みのノード列を歩いて、「どのフォントのどの文字がどの座標にあるか」「グルーやペナルティの実寸」を JSON にしてオーケストレータへ送る。
4. **出荷**：図などを本物の PDF ページとして書き出す（精密レンダー用）。

すべて LuaTeX の公式 API の範囲内で、ノードを「読む」ことはしても組版結果を「書き換える」ことはしません。

5.4 注入物 3：TeX マクロのシム (約 40 行)

LaTeX の一部の命令を、「元の動作をそのまま実行した上で、ついでに Lua へ記録する」形に包み直します。増分更新に必要な依存関係（どのブロックがどのラベルを参照しているか等）を知るためです。主なものを挙げます。

対象	目的	組版への影響
<code>\label</code> , <code>\ltx@label</code>	ラベル値の記録と即時定義	なし
<code>\ref</code> , <code>\pageref</code> , <code>\eqref</code>	「誰がどのラベルを参照するか」の記録	なし
<code>\cite</code> , <code>\@bibitem</code> 系	引用と文献番号の記録・即時解決	なし
<code>figure/table</code> 環境	フロートの中身と配置指定の捕捉	ライブ層のみ（印刷は無影響）
<code>\@starttoc</code>	目次を当方管理のファイルから読ませる	なし
ページ設定（余白原点ほか）	図の切り出しの原点合わせ	ライブ層のみ
フォント準備・数式代替フォント計測	起動時に一度だけ実行	なし（計測用の箱は捨てる）

フロート環境の包み直しも、箱の作り方（幅・字下げ復元・フォントリセット）は LaTeX 本体の `\@xfloat` と同一手順にしてあり、捕捉される箱は本来の出力ルーチンが受け取るはずだった箱と同じものです。

5.5 注入物 4：ページビルダーの「休眠」設定

TeX 内蔵のページビルダー（改ページ係）は、常駐運転では眠らせておきます。改ページはオーケストラ側で行うためです（理由は第 8 章）。眠らせ方は 3 つの設定の組み合わせです：「ページの高さを事実上無限大にする（ページが決して満杯にならない）」、「脚注の材料を横取りさせず行の位置に留める」、「ページの先頭に番兵の空箱を 1 つ置く（TeX が『ページはまだ空だ』と誤解して先頭の空白を捨てたり余計な空白を足したりするのを防ぐ）」。ユーザー原稿に `\newpage` などの強制改ページがあった場合に備えて、発火した出力ルーチンの中身をそっくり戻して「ここで改ページ」という印だけを残す安全網も入れてあります。

5.6 安全性について — `--shell-escape` の意味

C シムを Lua に読み込ませるため、`lualatex` は `--shell-escape` 付きで起動します。これは「TeX 文書が外部プログラムを起動してよい」という許可であり、信頼できる自分の文書を自分のマシンで組むためのローカル開発ツールとしての前提です。見知らぬ他人の `.tex` をそのまま流し込む用途は想定していません。

6 心臓部 1 — fork チェックポイント

6.1 fork は「TeX の完全なセーブデータ」である

TeX の内部状態は巨大です：数万のマクロ定義、文字種の表、ロード済みフォント、章や数式の番号カウンタ……。これをアプリの力で保存・復元するのは絶望的ですが、OS から見れば**ただのプロセスメモリ**です。

POSIX の `fork(2)` は動いているプロセスの完全な複製を作るシステムコールで、現代の OS では**コピーオンライト**（実際に書き換わったメモリページだけをあとから複製する方式）で実装されています。だから：

- fork 自体は一瞬 (実測 0.2 ミリ秒)。
- 複製のメモリ代は「その後書き換わった分」だけ。
- fork した親を眠らせて残せば、それが「その瞬間の TeX 完全状態の生きたセーブポイント」になる。

文書をブロック (≡段落・見出し・数式環境。空行で区切る) ごとに組版しながら、各境界で fork して親を残すと、次のようなプロセスの鎖ができます。

ckpt0 (前書き処理済み) → ckpt1 (ブロック 1 まで済み) → ckpt2 → … → ckptN (文書末尾まで済み)

ユーザーがブロック K を編集したら、(1) ckpt $_K$ 以降 (古い続きに基づく状態) を破棄し、(2) ckpt $_{K-1}$ に「新しいブロック K を組め」と命じます。子が組版して新しい ckpt $_K$ になります。プロセス起動もフォーマット読み込みもフォント読み込みも発生しません。これが数ミリ秒応答の正体です。

6.2 凍結のからくり

「プロセスを眠らせてセーブポイントにする」は、たった 1 行の TeX マクロと Lua の受信待ちで実現されています。TeX は `\directlua{tdom_wait()}` の内部で永久に停止し (ソケットの受信待ちなので CPU は消費しません)、fork で生まれた子だけが Lua から戻って組版を続けます。組版と報告を終えた子は、再び同じ待ち受けに入って次のセーブポイントになります。1 本のマクロループの上を、fork の世代が次々に流れていく構造です。

6.3 編集 1 回で起きること

1. ブラウザから「何文字目から何文字目をこう置き換えた」という差分が届く。
2. オーケストレータが本文をブロックに分け直し、内容ハッシュで前回と突き合わせて「変わったブロック」を特定する (1 ミリ秒未満)。
3. 変わったブロック以降のセーブポイントを破棄し、直前のセーブポイントに組版を命じる (fork → 組版 → 結果の JSON → 新セーブポイント昇格)。
4. **収束の自己検証**: 続きのブロックも 1 つ組み直してみ、出力と状態 (カウンタ類の出口値) が前回と完全一致したら、そこで止めてよいと判断する。一致しなければ (式番号がずれた等) 一致するまで先へ進む。「LaTeX をもう一回コンパイル」をブロック単位に局所化した形で、**停止判定そのものが検算**になっています。
5. ページ割り付けを再計算し、変わったページの描画命令だけをブラウザへ送る。

なお v3.2 からは、状態ベクトルに「直前の行の深さ」「見出し直後かどうか」というレイアウト上の連続状態も含まれており、段落間の空白に影響する変化もこの自己検証の網に掛かります。

6.4 プロセス数の管理

セーブポイント＝プロセスなので、巨大文書では数を絞ります（既定で最大 64 個）。間引かれた区間の編集は、少し手前のセーブポイントから数ブロック余分に組み直すだけです（1 ブロック数ミリ秒なので体感差はありません）。

7 心臓部 2 — 組版結果の収穫

7.1 本流の上で組み、写し取る

各ブロックは、TeX が文書を組み上げていく**本物のメイン垂直リストの上で**組版されます（第 5 章の「休眠」設定により、ページビルダーが横から切り取らないようにした上で）。組み終わったら、Lua デモンがたまったノード列を丸ごと写し取り、リストを空に戻してから凍結します。

この方式の要点は、**段落と段落の間の情報も全部本物になる**ことです。段落間の空白は「直前の行の深さ」に依存し、見出し直後の段落は字下げをしない——こうした前後関係は、プロセスのセーブポイント連鎖の中で自然に引き継がれます。実験により、写し取ったノード列は「素の `lualatex` が文書全体を一気に組んだ場合」と**バイト単位で一致**することを確認済みです。

7.2 run 分割の契約 — 座標を全部送らずに済む理由

写し取る情報は「行の箱」の中のグリフ（文字の字形）たちです。1 文字ずつ座標を送るとデータが膨れるため、次の契約を使います。

グリフの連なり（run）は、カーニングや空白（`kern / glue` のノード）が現れるたびに必ず分割する。すると run の内部にはもう位置調整が存在しないので、「**先頭の x 座標**」だけ送れば、あとはフォントの字送り幅を足すだけで TeX と同じ座標が再現できる。

ブラウザ側は TeX が使ったのと同じフォントファイルで描くので（次章）、字送り幅の出どころも同一です。これでペイロードが約 1 桁縮みます。

7.3 収穫される JSON の中身

ブロック 1 つの収穫結果には、行の箱（高さ・深さ・run の列）、実寸グルー（自然長と伸縮成分）、ペナルティ、脚注の中身（行位置に紐づく）、フロートの実体（箱の寸法と中身、配置指定）、使用フォントの実ファイルパス、記録されたラベルと参照、カウンタの出口値、が入っています。オーケストレータはこれを材料にページ割り付けと差分判定を行います。

8 心臓部 3 — ページ割り付け（規則書の移植）

8.1 なぜ Node 側で改ページするのか

増分更新の要は「編集していないページを触らない」ことです。改ページを TeX に任せると、1 ページ分の材料が確定するまで TeX を走らせる必要があり、ブロック単位のセーブポイントと相性が悪くなります。そこで改ページとページ合成だけをオーケストレータ側で行います。

ただし我流の割り付けでは本物とずれます。そこで v3.2 で、この部分を **TeX の原典プログラム（改ページの数式まで含む）** と **LaTeX の出力ルーチンの逐語的な移植**に置き換えました。具体的には：

- **改ページ判定**：TeX と同じ「改ページしてよい場所」の定義、同じコスト計算（空きの悪さ badness + ペナルティ。badness は原典と同じ整数演算）、同じ「最良候補を覚えておき、あふれた時点で確定」の手順。
- **ページ上端の空白**（`\topskip`）、**行の深さの繰り入れ**（`\maxdepth`）、**脚注の場所の先取り会計**（脚注が増えるほど本文の取り分が減る計算）も原典どおり。
- **フロート配置**：図表を「今のページの上か・下か・その場か・後回しか・専用ページか」に振り分ける LaTeX の判定式（回数制限・面積割合・順序保存のすべて）を条件式ごと移植。
- **ページ合成**：本文+脚注の区切り線（クラス定義を実測した寸法で再生）+脚注+上下フロートを、TeX と同じグルー分配（伸縮の次数まで考慮）で最終座標にする。

このファイルの中に**発明された寸法は 1 つもありません**。すべての定数は、起動時に本物の LaTeX から伸縮成分込みで実測して受け取ります。

9 描画層 — ブラウザで「印刷と同じ絵」を出す

9.1 フォントは TeX が使った実ファイルを配る

プレビューの文字は、TeX Live のディスクにあるフォントファイル（Latin Modern、原ノ味フォント等）**そのもの**をサーバーが配信し、ブラウザの `@font-face` として登録して描きます。TeX と同じ字形・同じ字送り幅が使われるので、run 契約（第 7 章）が成立します。

9.2 ブラウザの整形機能をすべて切る

ブラウザは放っておくと独自にカーニングやリガチャを再適用し、TeX の結果とずれます。そこで描画にはカーニング無効・リガチャ無効・字間調整ゼロを指定します。リガチャは TeX がすでに実行済みで、文字列には合字そのものが入っています。

9.3 数式の Type1 問題と「双子」フォント

古典的な LaTeX 数式のフォント (Computer Modern) は 1980 年代の Type1 形式で、ブラウザは表示できません。そこで公式後継の OpenType 版 (Latin Modern) を「双子」として使い、字形番号の対応表で写像します。それでも寸法が合わない大型記号 (大きな積分記号など) を含むブロックは、次の精密レンダー行きにします。

9.4 2 層戦略 — 即時グリフ+精密チャンク

TikZ の図やグラフのように「グリフでは描けないもの」(生の PDF 描画命令) は、2 段構えで表示します。

1. **即時層**: 打鍵直後は、まず文字ベースの近似を表示 (体感ゼロ遅延のため)。
2. **精密層**: 裏でレンダー子プロセスが該当ブロックを**本物の PDF ページ**として書き出し、SVG 画像に変換してプレビューに貼り込む (1~2 秒後に自動置換)。貼られる絵は本物の PDF の絵そのものなので、**原理的に印刷と同じ見た目**です。

レンダー子はセーブポイントから fork されるため、文書の途中の状態 (カウンタ・フォント・定義) を完全に引き継いだうえで図だけを出荷できます。

10 打鍵から画面まで — 時系列

キーを 1 つ打ってから画面が変わるまでの実測の内訳です (英語デモ文書の場合)。

段階	担当	時間の目安
差分の送信	ブラウザ	打鍵ごと即時
ブロック分割と差分特定	Node	1 ミリ秒未満
fork + 変更ブロックの組版	lualatex	4~14 ミリ秒
収穫 (ノード走査 → JSON)	Lua デーモン	数ミリ秒
ページ割り付け・描画命令の差分化	Node	1 ミリ秒前後
該当ページの描き替え	ブラウザ	数ミリ秒
合計 (打鍵 → 画面)		約 30 ミリ秒

式を 1 本挿入して以降の式番号が全部ずれるような編集でも、影響したブロックだけの再組版で約 50 ミリ秒です。日本語文書 (luatexja) は 1 編集あたり実測 5 ミリ秒前後。TikZ の図の編集は即時近似が数十ミリ秒、精密画像への置換が 1~2 秒です。

11 正しさの保証

「プレビュー＝本物の PDF」を主張だけで終わらせないため、二重の仕組みがあります。

■ (1) 収束の自己検証（常時・毎編集） 第 6 章のとおり、増分更新の停止判定そのものが「もう 1 ブロック組んで前回と完全一致するか」という検算です。

■ (2) 答え合わせ機（開発時・リリース時） tools/verify-layout.mjs は、同じ原稿を本エンジンと素の luatex の両方で処理し、全ページ・全行の位置を数値で突き合わせます。真値は PDF ファイルの中身（描画命令）を直接読んで取り出します。現在、同梱の全テンプレート・デモ文書・日本語実文書で全行が 0.02 bp（約 7 ミクロン = 髪の毛の 1/10 以下）以内で一致し、編集を繰り返した後の状態でも一致します（tools/verify-edits.mjs）。統合テストは 30 本すべて通過しています。

12 コードマップと API

12.1 リポジトリの地図

ファイル	行数	役割
server.js	269	HTTP/SSE サーバー。バックエンド選択、編集受付、配信
engine/checkpoint/engine-v3.js	約 1,700	オーケストレータ本体。差分・依存・収束・表示リスト
engine/checkpoint/daemon.lua	約 900	TeX 内デーモン。凍結・収穫・出荷
engine/checkpoint/pagebuilder.js	約 830	ページ割り付け（TeX 規則書の移植）
engine/checkpoint/tdomfork.c	71	fork の橋渡し C シム
engine/checkpoint/mathmap.js	186	数式レガシーフォント → 双子 OTF の対応表
engine/segmenter.js ほか	—	ブロック分割・ハッシュ差分（全世代共有）
engine/engine-lua.js, engine/luatex/	—	v1 バックエンド（比較用の参照実装）
engine/engine.js ほか	—	v0 内部エンジン（TeX 不要環境のフォールバック）
web/	約 1,000	ブラウザ側（エディタ・プレビュー・インスペクタ）
tools/	約 580	答え合わせ機・デバッグ道具
tests/	—	統合テスト 30 本
docs/	—	技術詳解 7 章＋本書＋レポート

12.2 3 世代のバックエンド

リポジトリには設計の進化が 3 つのバックエンドとして残っており、起動時に選べます（環境変数 TDOM_BACKEND）。

- **v0 internal** : TeX 不要の自前 JS 近似。アーキテクチャの参照実装兼フォールバック。
- **v1 luatex** : 編集のたびに変更ブロックだけを新しい luatex プロセスで組む方式（約 0.4 秒/編集）。fork が使えない環境の代替。

- **v3 checkpoint (既定)**：本書で説明した常駐 fork 方式。

12.3 HTTP API

ルート	内容
GET /doc	初期ロード一式（ソース・全ページ描画命令・ジオメトリ・フォント鍵）
POST /edit	範囲編集 <code>{start,end,text}</code> → 差分レポート+ページパッチ
POST /open	文書の開き直し（本文指定 or テンプレート指定）
GET /events	SSE。他ウィンドウ同期・図の精密画像の非同期差し替え
GET /font/...	TeX が実際に使ったフォントファイル
GET /chunk/...	図などの精密 SVG
GET /pdf	素の lualatex 2 回実行による正式 PDF
GET /dom	ブロック・依存・ページ対応の観測用 JSON
GET /templates	テンプレート一覧

13 動かし方と動作条件

- 必要なもの：Node.js、TeX Live (lualatex)、poppler (pdftocairo)、C コンパイラ (cc)。
npm の外部依存は**ゼロ**です。
- 起動：リポジトリで `npm start` → ブラウザで `http://127.0.0.1:4633`。
- OS：fork(2) を使うため macOS / Linux (Windows は WSL で可)。
- バックエンド切替：`TDOM_BACKEND=internal|lualatex|checkpoint`。

■**既知の未対応** 脚注のページまたぎ分割、二段組、欄外注 (marginpar)、figure/table 以外の独自フロート環境。いずれも検出時に診断を出し、だんまりで近似することはありません。

14 用語集

組版 (くみはん) 文字・図をどの位置にどんな間隔で置くかを決める作業。

ノード TeX 内部のデータ単位。文字・空白・罫線・ペナルティなどがリスト状につながる。

メイン垂直リスト 組み上がった行の箱が縦に積まれていく、文書の「本流」。

グルー 伸び縮みできる空白。自然長・伸び量・縮み量を持つ。

ペナルティ 「ここで改ページ (改行) すると減点いくつ」という印。負なら「むしろここで切れ」。

ページビルダー / 出力ルーチン 垂直リストからページを切り出し、脚注や図を合成する TeX / LaTeX の仕組み。本エンジンでは休眠させ、移植版がオーケストレータ側で同じ計算を行う。

フロート 図表を「近くの良い場所」に自動配置する LaTeX の仕組み。

fork / コピーオンライト プロセスの完全な複製を一瞬で作る OS 機能と、その実装方式。本エンジンのセーブポイントの実体。

チェックポイント 「ブロック k まで処理し終えた TeX の完全状態」を保持したまま眠っているプロセス。

ガレー 収穫された組版結果（行の箱と run の列）。活版印刷の「ゲラ」に由来。

run 途中に位置調整を含まないグリフの連なり。先頭座標だけで再現できる。

チャンク 図などを本物の PDF から切り出した精密 SVG 画像。

bp 長さの単位（ビッグポイント）。 $1\text{ bp} \doteq 0.353\text{ mm}$ 。PDF の内部単位。

本書自体も、本エンジンと同じ lualatex (TeX Live) で組版されています。